

# Django 教學 16

---

## 部署設定與正式環境

第 22～24 章

從最小範例到專案實作與驗收

# 本份學習路線

先完成 [15\\_安全與回歸測試整合](#)。

- **第 22 章：環境設定與機密**
- **第 23 章：Static、Media 與資料庫**
- **第 24 章：正式環境安全**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

# 第 22 章

---

## 環境設定與機密

整理環境變數清單，分辨可提交設定與機密，驗證 DEBUG 與 hosts。

# 本章的操作環境與成果

部署練習使用 LearnBoard 或 LearnMart 副本；LearnJournal 僅作延伸示例。

**完成成果：** 整理環境變數清單，分辨可提交設定與機密，驗證 DEBUG 與 hosts。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。  
進階頁可回查，但所有基本驗收需完成。

## 22-1 為什麼做成共用單元？

三個專案都會遇到同一批 production 問題：

- SECRET\_KEY / DEBUG / ALLOWED\_HOSTS
- static / media
- database
- HTTPS / cookies / security headers
- process / reverse proxy
- logging / monitoring
- migrations / backup / rollback
- CI / release checklist

不需要複製三份。

## 22-2 本機與 production 的差別

本機常見：

```
runserver + SQLite + DEBUG=True + local files
```

production 常見：

```
reverse proxy  
  ↓ HTTPS  
WSGI/ASGI process manager  
  ↓  
Django  
  ↓  
PostgreSQL / shared cache / object storage
```

## 22-3 這個單元不綁供應商

你可以把概念套到：

- VM
- Docker / container platform
- PaaS
- school server
- managed database

真正要學的是責任邊界與驗證方式。

## 22-4 部署練習的起點與配套手冊

從已完成的 LearnBoard 或 LearnMart 練習分支開始，先確認測試通過。

- 環境變數只有在 settings 實際讀取後才會生效。
- 先在本機的類正式設定驗證，不直接改動正在服務的網站。
- 原部署 workbook 以 LearnJournal 示範部分 env／PostgreSQL 設定。
- 本課依第 22～24 章把相同設定套到所選專案；LearnJournal 只作參考。

部署 [workbook](#) 的章號另見本目錄對照索引。

## 22-5 哪些值不該硬編碼？

- SECRET\_KEY
- database password
- SMTP password
- object-storage credentials
- API keys
- production host names

這些值屬於部署環境，不屬於教材 repository。

## 22-6 Classroom default vs production override

LearnJournal 現在示範：

```
SECRET_KEY = os.getenv(  
    "DJANGO_SECRET_KEY",  
    "django-insecure-...classroom-only...",  
)  
DEBUG = os.getenv("DJANGO_DEBUG", "1") == "1"
```

本機 clone 可直接跑；production 必須 override。

## 22-7 將環境設定接到所選專案

教學新增 | 在練習副本的 `config/settings.py` 替換同名設定

```
import os
from django.core.exceptions import ImproperlyConfigured

DEBUG = os.environ.get("DJANGO_DEBUG", "1") == "1"
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY", "")
if not SECRET_KEY:
    if not DEBUG:
        raise ImproperlyConfigured("請設定 DJANGO_SECRET_KEY")
    SECRET_KEY = "django-insecure-local-practice-only"
ALLOWED_HOSTS = [h.strip() for h in os.environ.get(
    "DJANGO_ALLOWED_HOSTS", "localhost,127.0.0.1").split(",")
    if h.strip()]
```

先確認值確實被 settings 讀取，再做下一頁的環境變數操作。

## 22-8 讀環境變數

macOS/Linux :

```
export DJANGO_DEBUG=0  
export DJANGO_SECRET_KEY='replace-me'
```

PowerShell :

```
$env:DJANGO_DEBUG="0"  
$env:DJANGO_SECRET_KEY="replace-me"
```

驗證 :

```
uv run python manage.py shell -c "from django.conf import settings; print(settings.DEBUG)"
```

## 22-9 不要提交 `.env`

`.env` 可以是本機便利工具，但它通常包含 secrets。

Git 原則：

|                           |                 |
|---------------------------|-----------------|
| <code>.env</code>         | → ignore        |
| <code>.env.example</code> | → 可提交，只放欄位名稱與假值 |

真正 production secrets 交給平台 secret manager / environment config。

## 22-10 ALLOWED\_HOSTS

DEBUG=False 時要明確列允許 host：

```
DJANGO_ALLOWED_HOSTS=example.com,www.example.com
```

這不是 CORS；它是 Host header 防護的一部分。

## 22-11 Settings split 要不要做？

小專案可以先用 environment variables。

專案變大後可拆：

```
settings/  
  base.py  
  dev.py  
  production.py
```

重點不是檔案數，而是避免 dev/prod 差異散落在程式碼各處。

## 22-12 本章檢核

1. SECRET\_KEY 為什麼不能提交？
2. `.env.example` 可以放真密碼嗎？
3. ALLOWED\_HOSTS 和 CORS 是同一件事嗎？
4. 何時值得拆 settings module？

## 22-13 開發 settings 不等於 production baseline

目前 LearnMart 刻意是 classroom config

```
SECRET_KEY = "django-insecure-..."  
DEBUG = True  
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
```

正式部署還需環境變數、HTTPS、secure cookies、HSTS、logging、production database、media strategy 等。

## 第 22 章實作與離堂檢核

**任務：** 整理環境變數清單，分辨可提交設定與機密，驗證 DEBUG 與 hosts。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

**配套練習：** 部署與維運 [workbook](#)。手冊保留原章號，對照表列出本課位置。

# 第 23 章

## Static、Media 與資料庫

說明 static、media、database 的部署責任，完成資產與資料搬遷檢核。

## 本章的操作環境與成果

部署練習使用 LearnBoard 或 LearnMart 副本；LearnJournal 僅作延伸示例。

**完成成果：**說明 static、media、database 的部署責任，完成資產與資料搬遷檢核。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。  
進階頁可回查，但所有基本驗收需完成。

## 23-1 `runserver` 幫你做了很多事

開發環境 `DEBUG=True` 時 static 看起來「自己就能用」。

production 不應依賴 `runserver`。

## 23-2 collectstatic

```
uv run python manage.py collectstatic --noinput
```

它把各 app / `STATICFILES_DIRS` 的 static 收集到：

```
STATIC_ROOT
```

再交給 web server / CDN / storage backend 提供。

## 23-3 Media 不等於 Static

LearnMart 的商品圖片是 media。

不要把使用者上傳檔跟程式版本綁在 container image 裡。

production 常見做法：

- persistent disk
- object storage
- CDN 前置

## 23-4 上傳安全

至少思考：

- content type / extension 不能完全相信 client
- 檔案大小限制
- image decode / malware scanning
- private vs public media
- 不讓 upload path 直接執行程式碼

## 23-5 SQLite 到 PostgreSQL

SQLite 非常適合教學與單機 demo。

production 多使用者服務通常需要：

- concurrent writes
- server DB
- backup / replication
- 權限隔離
- richer indexing / FTS

LearnJournal 延伸教材已示範用環境變數切 PostgreSQL。

## 23-6 切 DB 不等於搬資料

改設定後：

```
uv run python manage.py migrate
```

只建立 schema；舊 SQLite 資料不會自動出現在 PostgreSQL。

資料搬遷是另一個 migration/import 工作。

## 23-7 `dumpdata` / `loaddata` 的定位

教學小資料可示範：

```
uv run python manage.py dumpdata --indent 2 > data.json
uv run python manage.py loaddata data.json
```

但大型 production DB backup 應使用資料庫原生工具與 managed backup。

## 23-8 本章檢核

1. static / media 的來源與生命週期差在哪？
2. `collectstatic` 會搬使用者上傳檔嗎？
3. 切換 PostgreSQL 後為什麼資料是空的？
4. production backup 為什麼不能只靠 `dumpdata` ？

## 第 23 章實作與離堂檢核

**任務：** 說明 static、media、database 的部署責任，完成資產與資料搬遷檢核。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

**配套練習：** 部署與維運 [workbook](#)。手冊保留原章號，對照表列出本課位置。

# 第 24 章

---

## 正式環境安全

在類正式設定下執行部署檢查，逐項記錄風險與修正結果。

## 本章的操作環境與成果

部署練習使用 LearnBoard 或 LearnMart 副本；LearnJournal 僅作延伸示例。

**完成成果：** 在類正式設定下執行部署檢查，逐項記錄風險與修正結果。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。  
進階頁可回查，但所有基本驗收需完成。

## 24-1 先跑 Django 自己的檢查

```
DJANGO_DEBUG=0 \  
DJANGO_SECRET_KEY='a-long-random-secret' \  
DJANGO_ALLOWED_HOSTS='example.com' \  
uv run python manage.py check --deploy
```

不要只看「System check identified no issues」；讀懂每個 warning 在保護什麼。

## 24-2 DEBUG 必須關

`DEBUG=True` 可能暴露：

- stack trace
- settings/context
- SQL/debug 資訊
- filesystem path

production error page 應對使用者簡潔，詳細資訊寫 log。

## 24-3 HTTPS 與 secure cookies

HTTPS 不只是登入頁。

Production 通常需要：

```
SESSION_COOKIE_SECURE = True  
CSRF_COOKIE_SECURE = True
```

若 TLS 在 reverse proxy 終止，還要正確設定 proxy header，避免 Django 誤判 request scheme。

## 24-4 CSRF / XSS / Clickjacking

這些不是「前面課過就結束」：

- CSRF middleware 不要隨便關
- template autoescape 不要濫用 `safe`
- user HTML 要 sanitize
- `X_FRAME_OPTIONS` / CSP 依需求設定
- upload 內容也要視為不可信

## 24-5 Security headers

LearnJournal 示範：

```
SECURE_CONTENT_TYPE_NOSNIFF = True  
X_FRAME_OPTIONS = "DENY"
```

真正 production 還要依架構評估 HSTS、CSP、Referrer-Policy 等。

## 24-6 Rate limiting / brute force

Django core 不替你的所有 endpoint 做 rate limit。

要針對：

- login
- password reset
- registration
- comments/reviews
- expensive search

在 proxy、middleware 或專用套件層處理。

## 24-7 本章檢核

1. `check --deploy` 是自動修復嗎？
2. TLS 在 proxy 終止時 Django 為什麼可能不知道原始 request 是 HTTPS？
3. `safe` filter 為什麼是安全邊界？
4. 哪些 endpoint 特別需要 rate limit？

## 24-8 `check` 與 `check --deploy` 不同

```
uv run python manage.py check
```

檢查一般 project configuration。

```
uv run python manage.py check --deploy
```

會對 production security settings 提出警告；教學版預期會出現多項警告，不代表要在本地課堂設定盲目消除所有項目。

## 第 24 章實作與離堂檢核

**任務：** 在類正式設定下執行部署檢查，逐項記錄風險與修正結果。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

**配套練習：** 部署與維運 `workbook`。手冊保留原章號，對照表列出本課位置。

# 本份完成與後續

下一份：[17\\_服務運行與發布維護](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。